

第七章 可编程逻辑器件

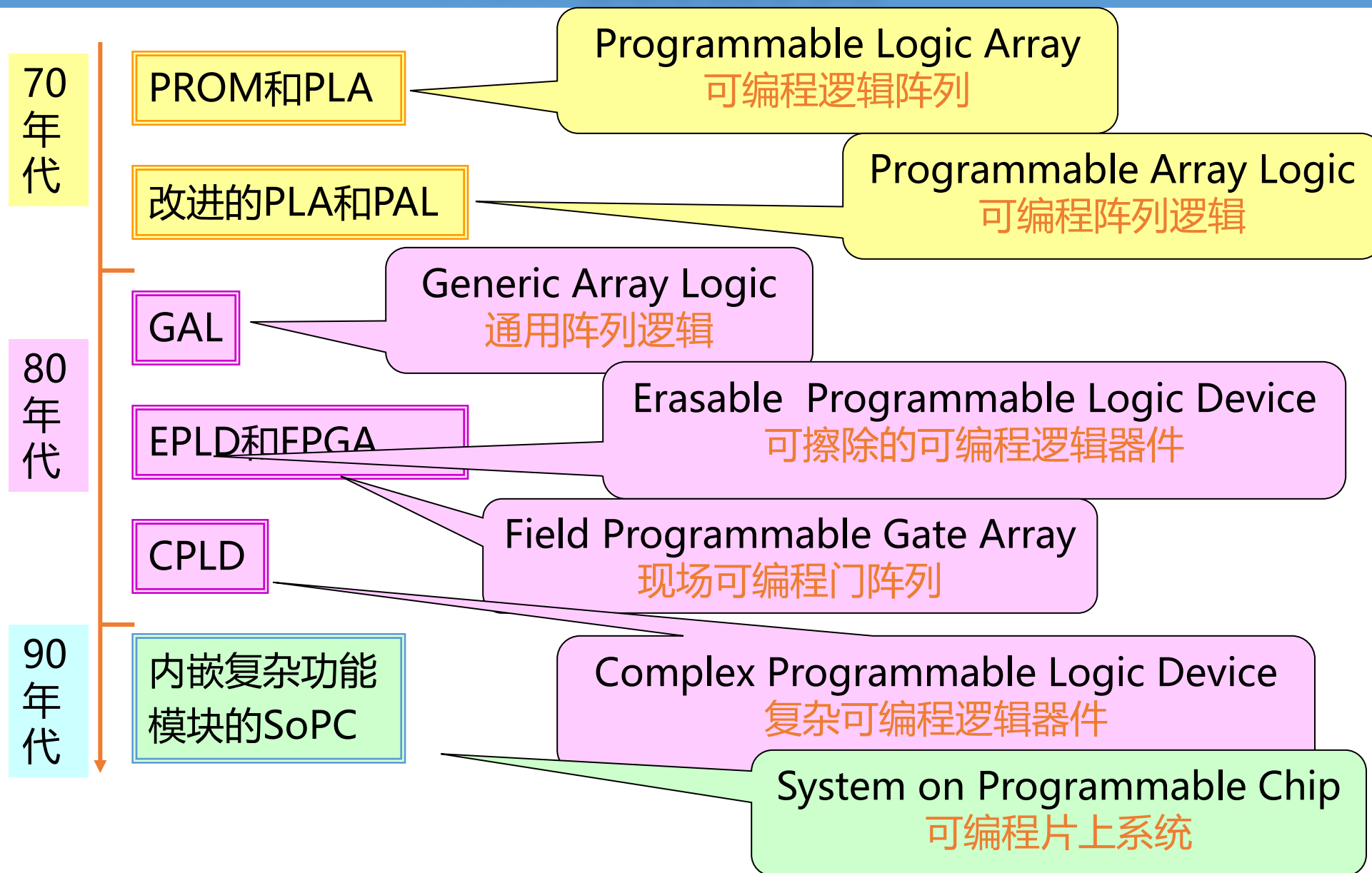
- 7.1 存储器及其在可编程逻辑实现方面的应用器
- 7.2 PLA、PAL、GAL
- 7.3 EPLD
- 7.4 CPLD
- 7.5 FPGA

可编程逻辑器件

PLD (Programmable Logic Device) :
主要由**逻辑单元、互连线单元、输入/输出单元**组成，各单元的功能及相互连接关系都可经编程设置。借助EDA (Electronic Design Automation) 工具软件，**用户自己可设计、修改PLD所完成的逻辑功能。**

优点：速度快、灵活性好、集成度高，是当前数字系统设计的主流器件。

PLD发展历史



PLD开发系统

硬件：计算机和编程器

系统可编程（ISP）器件不需要专门的编程器。

软件：硬件描述语言和工具软件

VHDL、Verilog HDL、
AHDL、ABEL等

Quartus II、MAX+plusII、
ISE、Foundation、
ispEXPERT等。

7.1.0 存储器

半导体存储器是一种能够存储和管理大量二值信息（二值数据）的半导体器件。存储器的结构和功能使其可用于逻辑函数的可编程实现。

分类

- 按制造工艺分为：双极型和MOS型
MOS电路尤其是CMOS电路具有功耗低、集成度高的优点，因此目前大容量的存储器都是采用MOS工艺制作。
- 按存取功能分为：只读存储器(READ-ONLY MEMORY, ROM)和随机存取存储器(RANDOM - ACCESS MEMORY, RAM)

7.1.1 只读存储器(一)

只读存储器在正常工作时，它存储的数据是固定不变的，只能从中读取数据，不能快速地随时修改或重新写入数据。

只读存储器的优点：电路结构简单，断电以后数据不会丢失。

只读存储器的缺点：只适用于存储固定数据的场合。

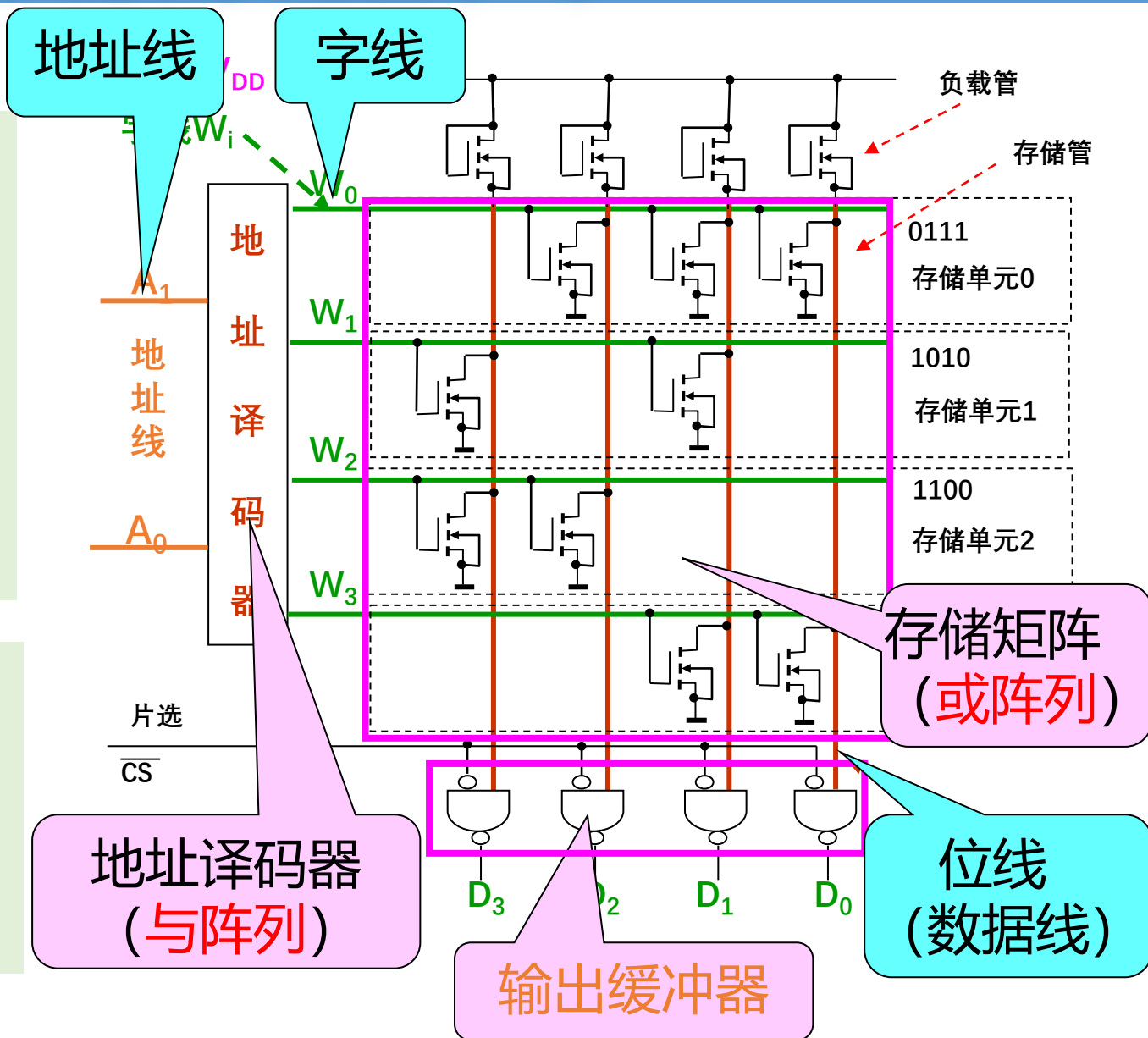
7.1.1 只读存储器(二)

ROM的结构

由若干**存储单元** (**字**)组成, 每一单元存储了 **m** 个**二进制位**。

输入ROM的为 **n** 条**地址线** (A_i), 地址线经**地址译码器**给出 **2^n** 条**字线**, 每条**字线** (W_k) 寻址一个**存储单元**。

被寻址的存储单元通过 **m** 条**位线** (D_j) 将存储的0、1信息送出ROM。

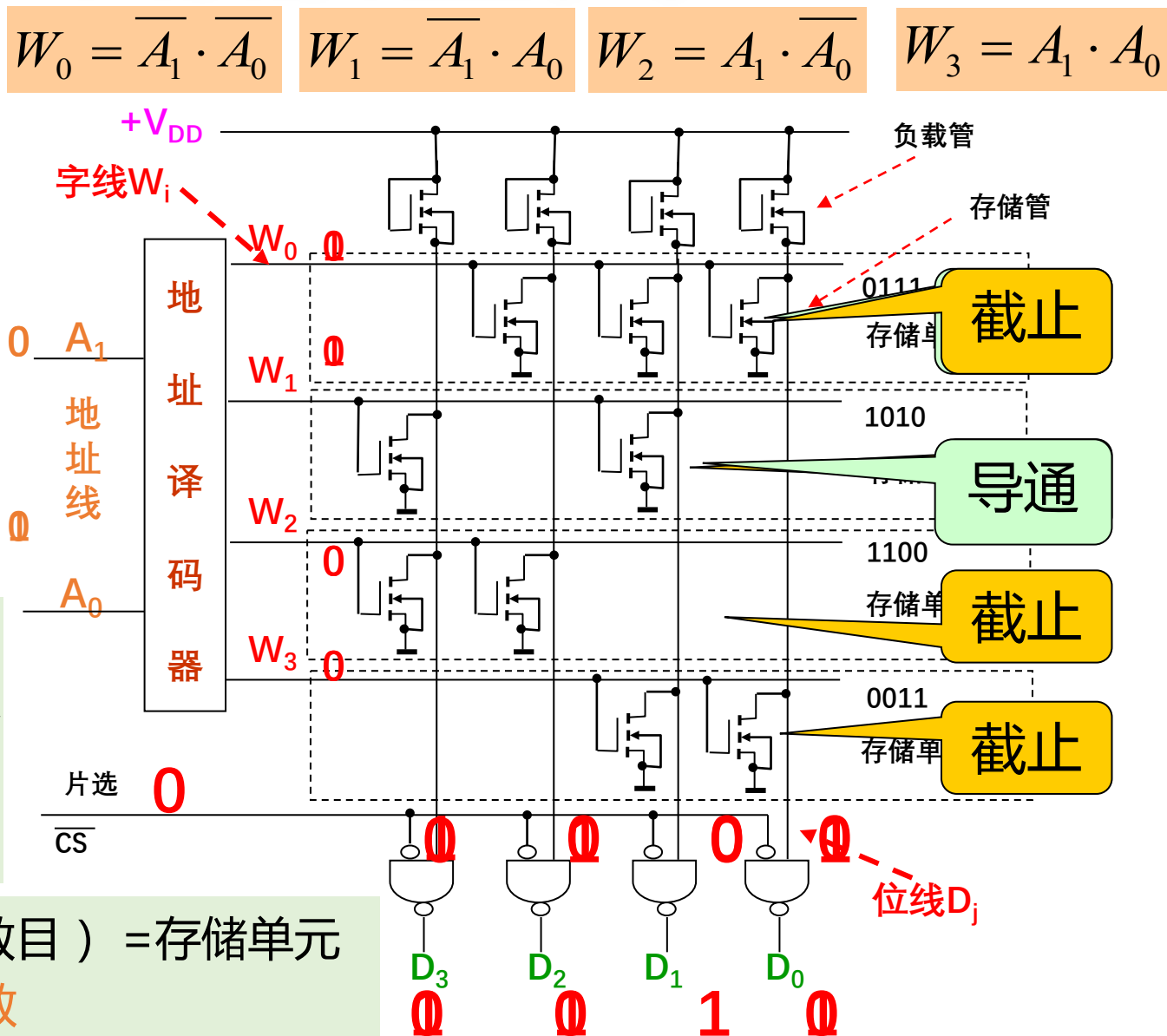


7.1.1 只读存储器(三)

地 址	数 据
$A_1 A_0$	$D_3 D_2 D_1 D_0$
0 0	0 1 1 1
0 1	1 0 1 0
1 0	1 1 0 0
1 1	0 0 1 1

字线与位线的交叉处接有MOS管的相当于存 '1'，没有MOS管的相当于存 '0'。

存储器的容量（存储位的数目）= 存储单元数 × 每单元位数 = 字数 × 位数



7.1.1 只读存储器(四)

地 址	数 据
$A_1 A_0$	$D_3 D_2 D_1 D_0$
0 0	0 1 1 1
0 1	1 0 1 0
1 0	1 1 0 0
1 1	0 0 1 1

$$D_3 = \overline{A_1} A_0 + A_1 \overline{A_0} = W_1 + W_2$$

$$D_2 = \overline{A_1} \cdot \overline{A_0} + A_1 \overline{A_0} = W_0 + W_2$$

$$D_1 = \overline{A_1} \cdot \overline{A_0} + \overline{A_1} A_0 + A_1 A_0 = W_0 + W_1 + W_3$$

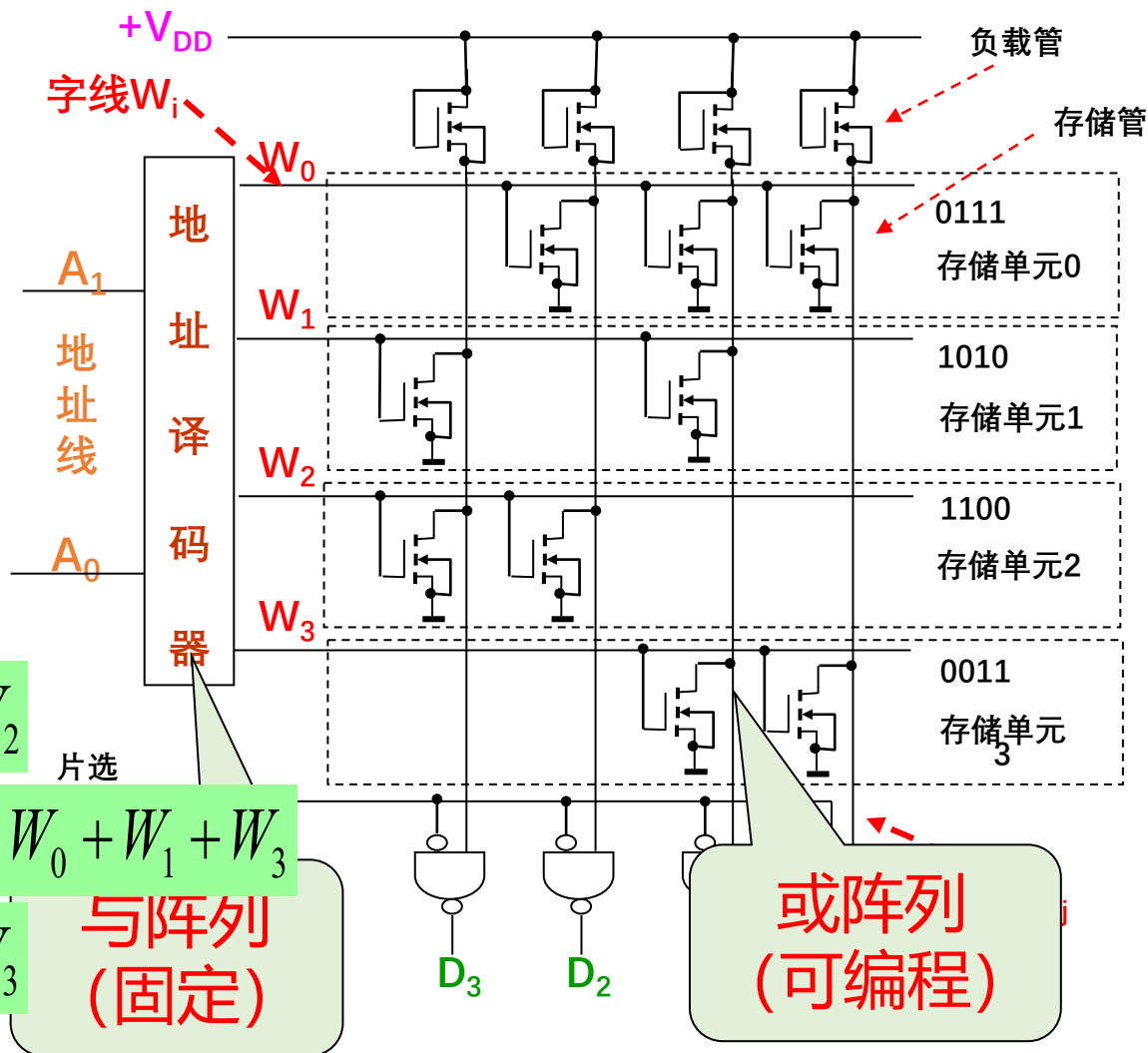
$$D_0 = \overline{A_1} \cdot \overline{A_0} + A_1 A_0 = W_0 + W_3$$

$$W_0 = \overline{A_1} \cdot \overline{A_0}$$

$$W_1 = \overline{A_1} \cdot A_0$$

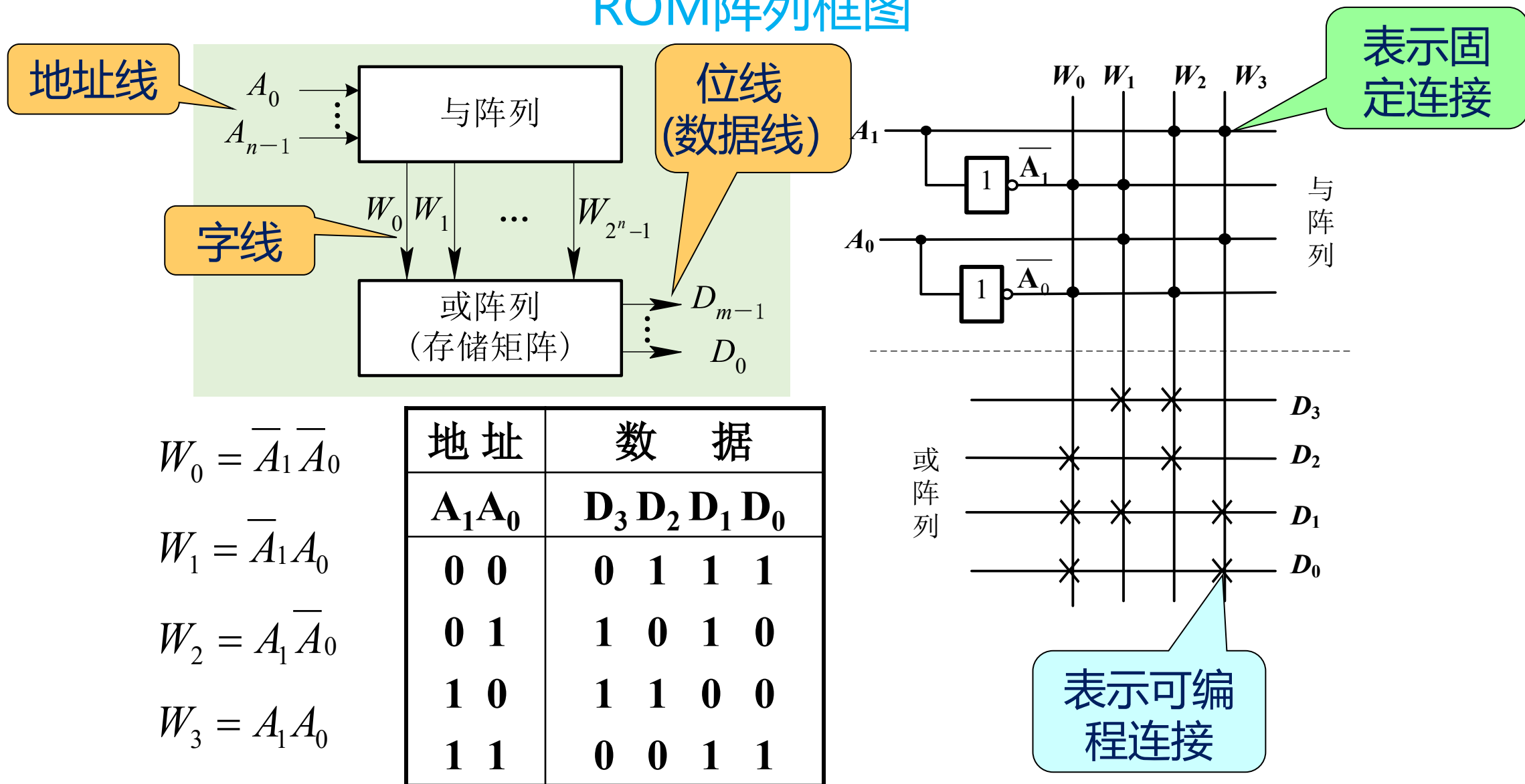
$$W_2 = A_1 \cdot \overline{A_0}$$

$$W_3 = A_1 \cdot A_0$$



7.1.1 只读存储器(五)

ROM阵列框图



7.1.1 只读存储器(六)

用存储器实现组合逻辑函数

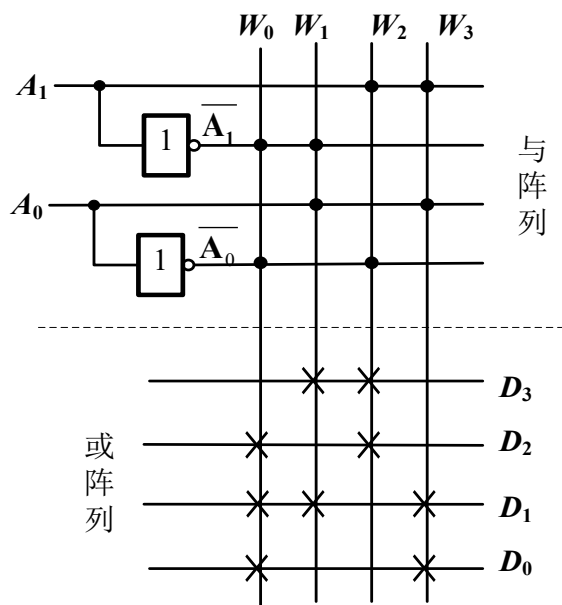
地 址	数 据
$A_1 A_0$	$D_3 D_2 D_1 D_0$
0 0	0 1 1 1
0 1	1 0 1 0
1 0	1 1 0 0
1 1	0 0 1 1

在ROM的数据表中，如果将输入地址 A_1 、 A_0 看成两个输入逻辑变量，而将数据输出 D_3 、 D_2 、 D_1 、 D_0 看成一组输出逻辑变量，则 D_3 、 D_2 、 D_1 、 D_0 就是 A_1 、 A_0 的一组逻辑函数，ROM的数据表就是这一组多输出组合逻辑函数的真值表，因此该ROM可以实现表中的四个函数（ D_3 、 D_2 、 D_1 、 D_0 ），其表达式为：

$$D_3 = \overline{A_1}A_0 + A_1\overline{A_0} \quad D_2 = \overline{A_1} \cdot \overline{A_0} + A_1\overline{A_0} \quad D_1 = \overline{A_1} \cdot \overline{A_0} + \overline{A_1}A_0 + A_1A_0 \quad D_0 = \overline{A_1} \cdot \overline{A_0} + A_1A_0$$

ROM可作为PLD实现多输出组合逻辑函数。

7.1.1 只读存储器(七)



从ROM结构看，其地址译码器形成了输入地址变量的全部最小项，即每一条字线对应输入地址变量的一个最小项，而每一位数据输出又都是若干个最小项之和，因此任何形式的组合逻辑函数均可以通过向ROM中写入相应的数据来实现。

用具有n位输入地址，m位数据输出的ROM可以获得一组最多m个的任何形式的n变量组合逻辑函数，只要根据需实现逻辑函数的最小项表达式向ROM中写入相应的数据。

此原理同样适用于RAM

7.1.3 随机存储器(一)

- 随机存储器又叫**读/写存储器**，简称**RAM**。在RAM工作时既可以随时从任何一个指定的地址取出（**读出**）数据，也可以随时将数据存入（**写入**）任何指定地址的存储单元中去。

优点：读写方便，使用灵活。

缺点：存在数据易失性，一旦断电所存储的数据便会丢失，不利于数据长期保存。

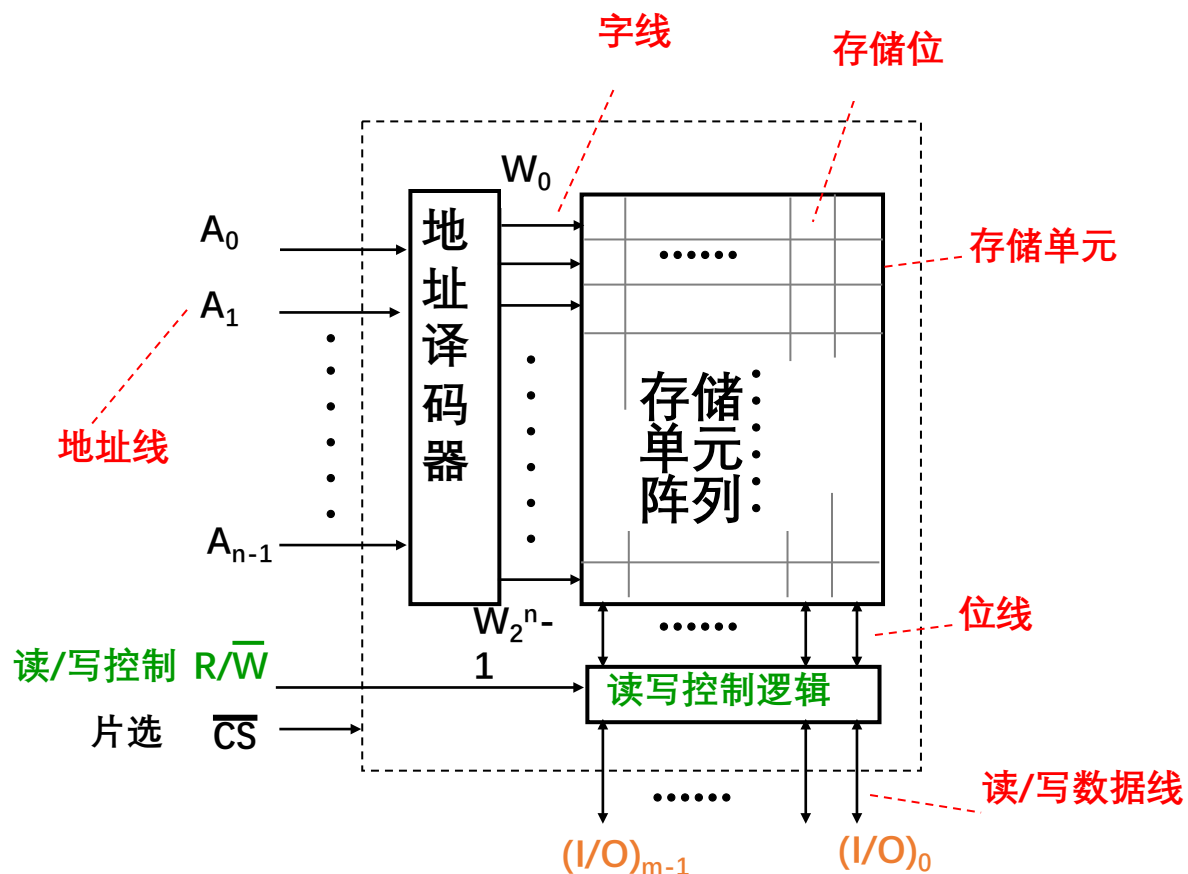
- 按存储单元的特性分为：

SRAM： **静态**随机存储器。存取速率一般相对较快。

DRAM： **动态**随机存储器。集成度会相对较高。

7.1.3 随机存储器(二)

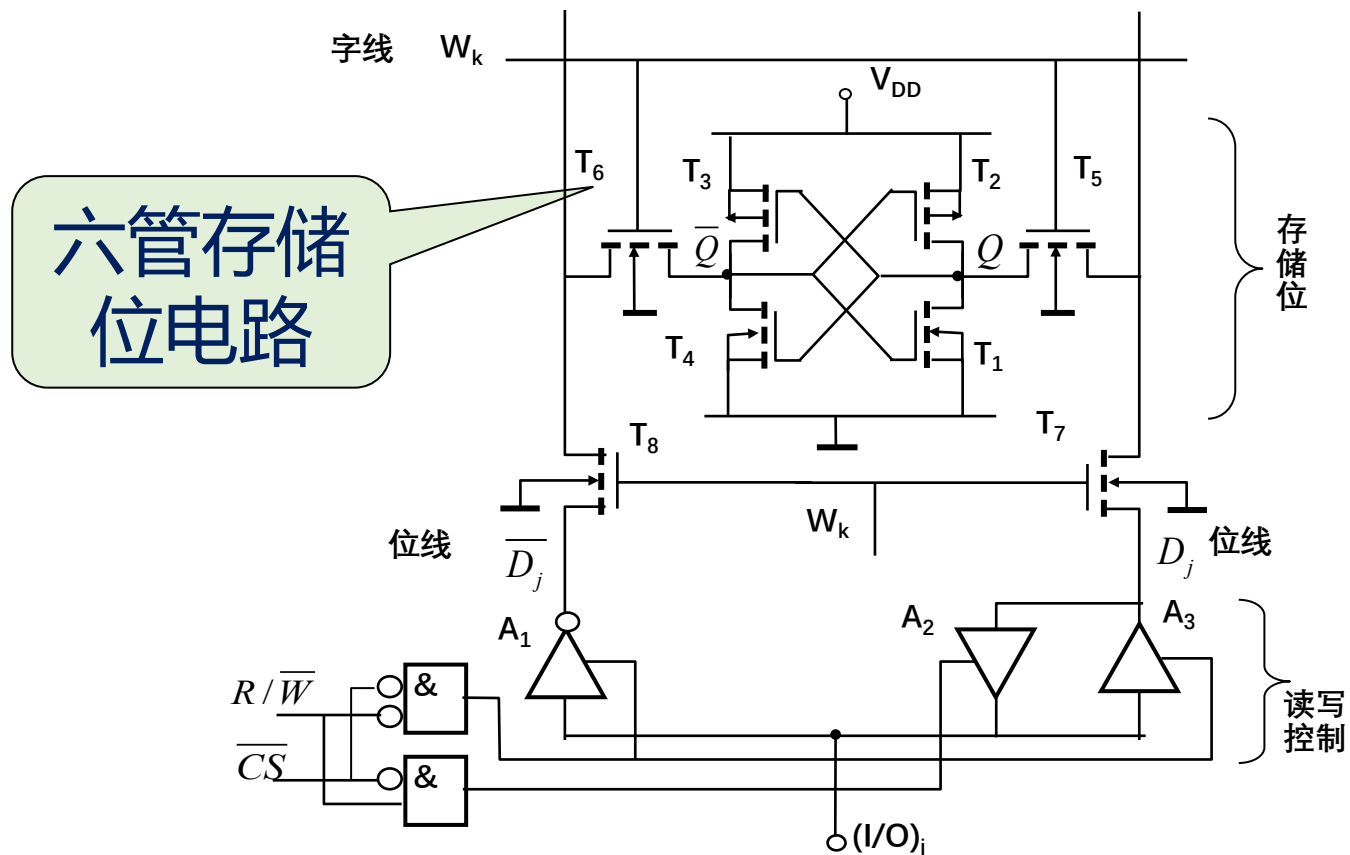
RAM的结构



- RAM的结构与ROM类似，主要由地址译码器和存储单元阵列构成。地址译码器给出 n 位地址变量的全部最小项 W_k ($k=0 \sim 2^n-1$), 存储单元阵列完成可编程或运算。
- RAM可被认为是一种与运算固定、或运算可编程的逻辑器件。

7.1.3 随机存储器(三)

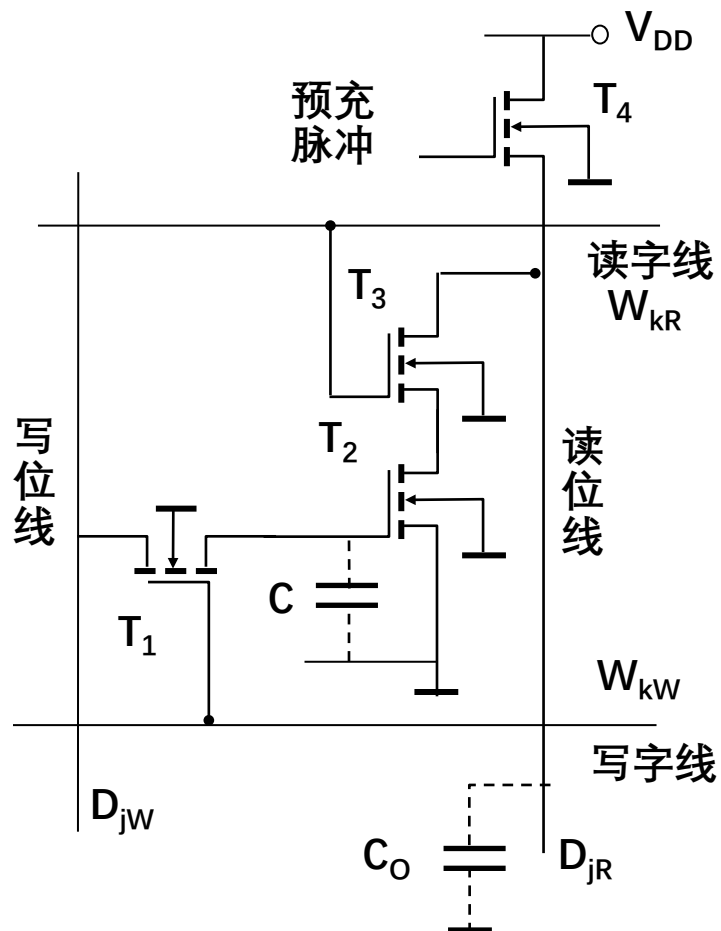
(1) 静态随机存储器中的存储位



静态存储位在静态触发器的基础上附加门控管而构成，它靠触发器的自保功能存储数据。

7.1.3 随机存储器(四)

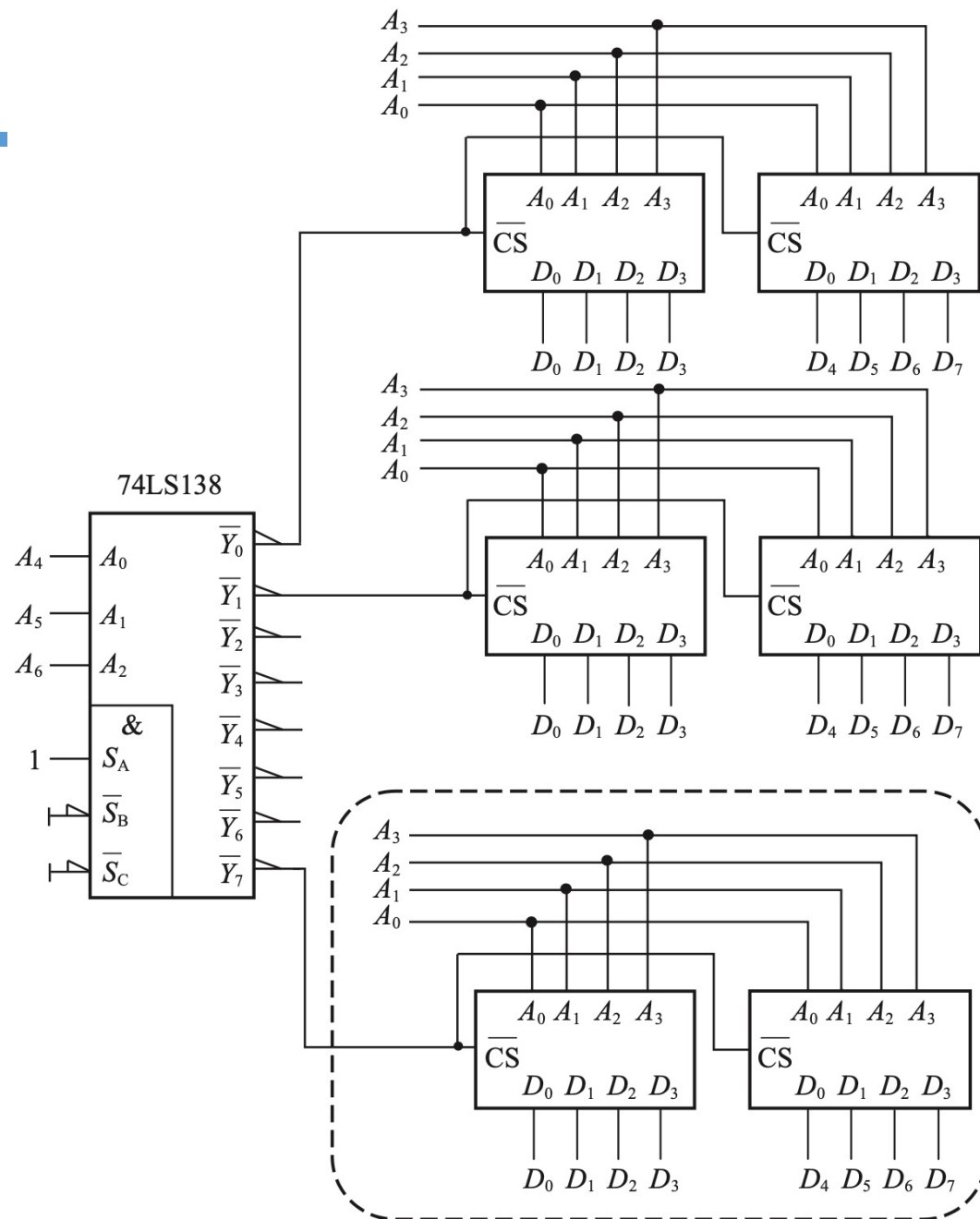
(2) 动态随机存储器中的存储位



- 动态存储位利用MOS管栅极电容可以存储电荷的原理制成。电路结构可以做得非常简单，因此被普遍应用于大容量、高集成度的RAM中。
- **缺点：** 由于MOS管栅极电容的容量很小（通常仅为几皮法），且存在漏电流，所以存储在电容上的电荷保存的时间有限，需要及时补充电荷，这种操作称为**刷新**或**再生**。

补充：扩展存储器容量

- 当需要扩展字的宽度时，只需将多片相同存储器的地址以及片选各自并联即可；当需要扩展字数时，可外加地址译码器或译码电路。



7.1.4 用存储器实现逻辑处理(一)

- 从逻辑运算的角度看存储器是一种与运算固定、或运算可编程的逻辑器件。
- 存储器中的地址译码器产生地址变量的全部最小项，而存储单元中写入的数据决定了每一位数据输出都是若干个最小项之和，而任何一个逻辑函数都可以写成若干个输入变量的最小项之和的标准形式。
- 任何形式的逻辑函数均可以通过向存储单元写入相应的数据来实现，既由存储单元中写入的数据决定逻辑函数的构成。

7.1.4 用存储器实现逻辑处理(二)

(1) 存储器实现组合逻辑

- 根据逻辑函数的输入、输出变量数目，确定ROM或RAM的容量，选择合适的ROM或RAM。

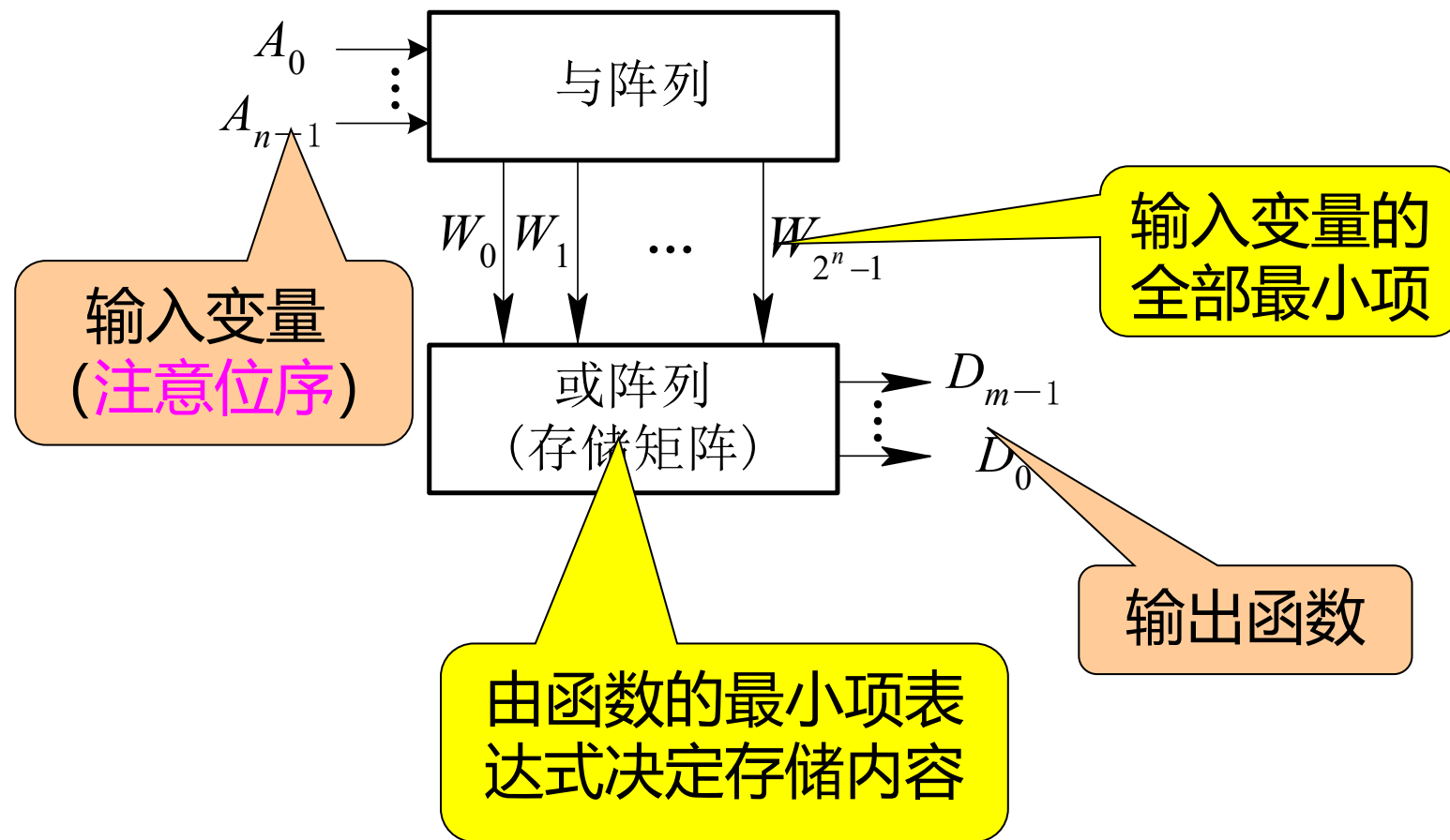
当用存储器实现 n 个逻辑变量、 m 个输出的组合逻辑函数时需要用 n 位地址、 2^n 个存储单元、每个单元有 m 位的ROM或RAM。

- 写出所要实现的逻辑函数的最小项之和的标准表达式，画出ROM的阵列图。

存储器的各地址线 $A_i (i=n-1 \sim 0)$ 与函数的各个输入变量依次连接。由存储器的输出数据线 D_j 得到第 j 个逻辑函数($j=m-1 \sim 0$)。

- 根据阵列图对ROM进行编程。

7.1.4 用存储器实现逻辑处理(三)



7.1.4 用存储器实现逻辑处理(四)

例7.1.1：用ROM实现四位自然二进制码与循环码的转换电路。

需要4位地址、4位数据的ROM(容量为16×4)实现此转换电路。设四位二进制码为DCBA，循环码为WXYZ。

$$W = \sum_m(8,9,10,11,12,13,14,15)$$
$$X = \sum_m(4,5,6,7,8,9,10,11)$$
$$Y = \sum_m(2,3,4,5,10,11,12,13)$$
$$Z = \sum_m(1,2,5,6,9,10,13,14)$$

二进制码				循环码				
D	C	B	A	W	X	Y	Z	
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	3
0	0	1	1	0	0	1	0	2
0	1	0	0	0	1	1	0	6
0	1	0	1	0	1	1	1	7
0	1	1	0	0	1	0	1	5
0	1	1	1	0	1	0	0	4
1	0	0	0	1	1	0	0	C h
1	0	0	1	1	1	0	1	D h
1	0	1	0	1	1	1	1	F h
1	0	1	1	1	1	1	0	E h
1	1	0	0	1	0	1	0	A h
1	1	0	1	1	0	1	1	B h
1	1	1	0	1	0	0	1	9
1	1	1	1	1	0	0	0	8

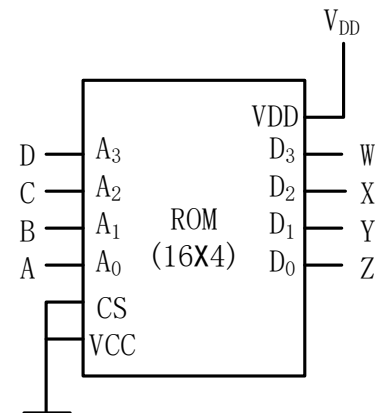
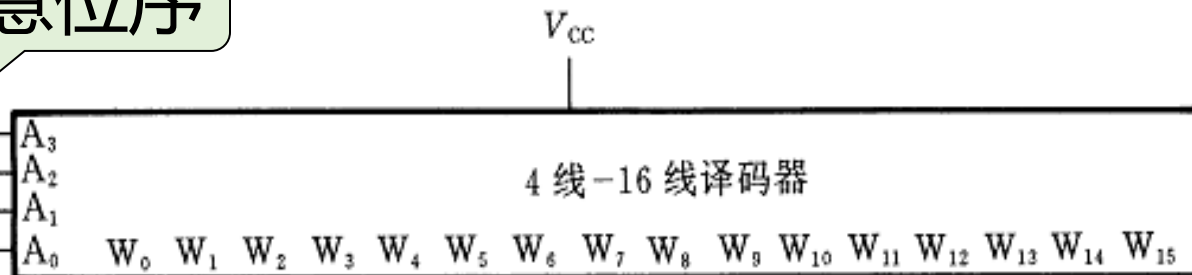
ROM
存储
单元
的
内
容

A₃A₂A₁A₀ D₃D₂D₁D₀

7.1.4 用存储器实现逻辑处理(五)

注意位序

D
C
B
A



$W =$

$\sum_m(8,9,10,11,12,13,14,15)$

$X =$

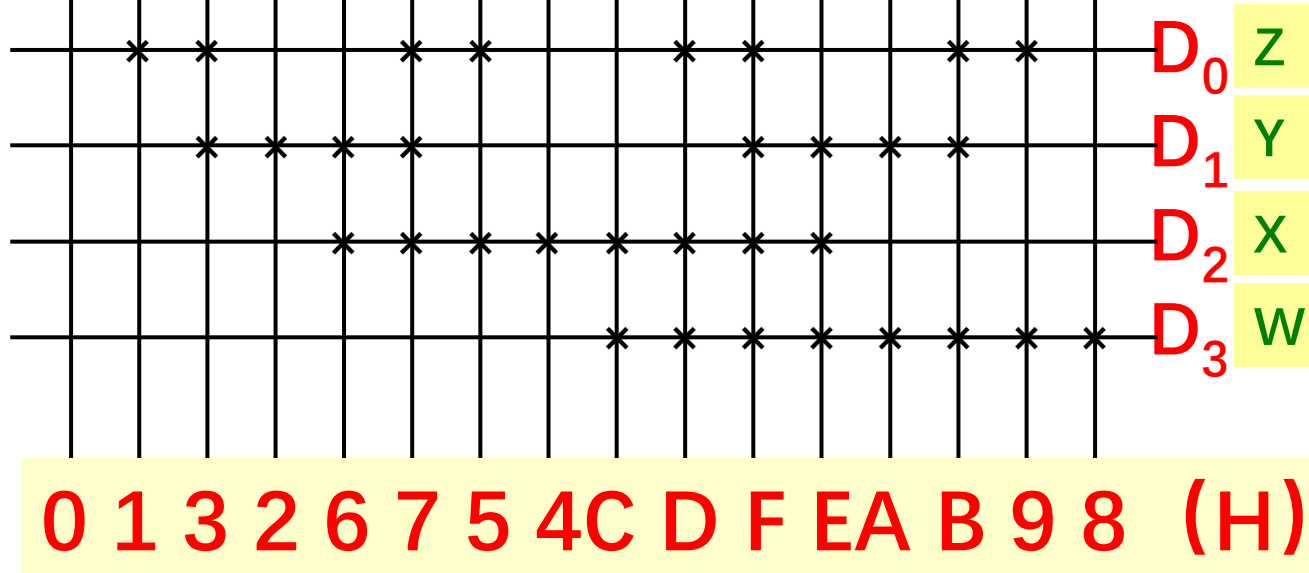
$\sum_m(4,5,6,7,8,9,10,11)$

$Y =$

$\sum_m(2,3,4,5,10,11,12,13)$

$Z =$

$\sum_m(1,2,5,6,9,10,13,14)$



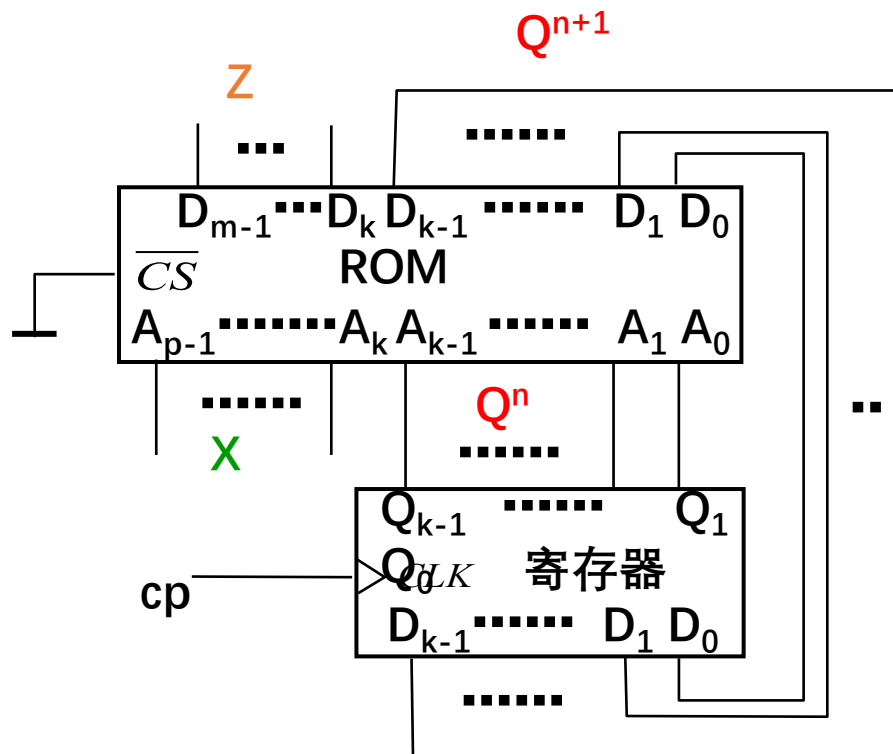
7.1.4 用存储器实现逻辑处理(六)

- 由存储器实现的组合逻辑电路**不会出现逻辑冒险**，因为不存在信号的多路径传输。存储器内部的电路设计可保证输出信号的稳定性。
- 由存储器实现的组合逻辑电路仍**有可能出现功能冒险**，因为功能冒险是由于多个输入信号的不同步而产生的。当多个地址变量出现变化的时刻偏差大于存储器的读取时间，功能冒险就存在，输出信号上可能出现毛刺噪声。

7.1.4 用存储器实现逻辑处理(七)

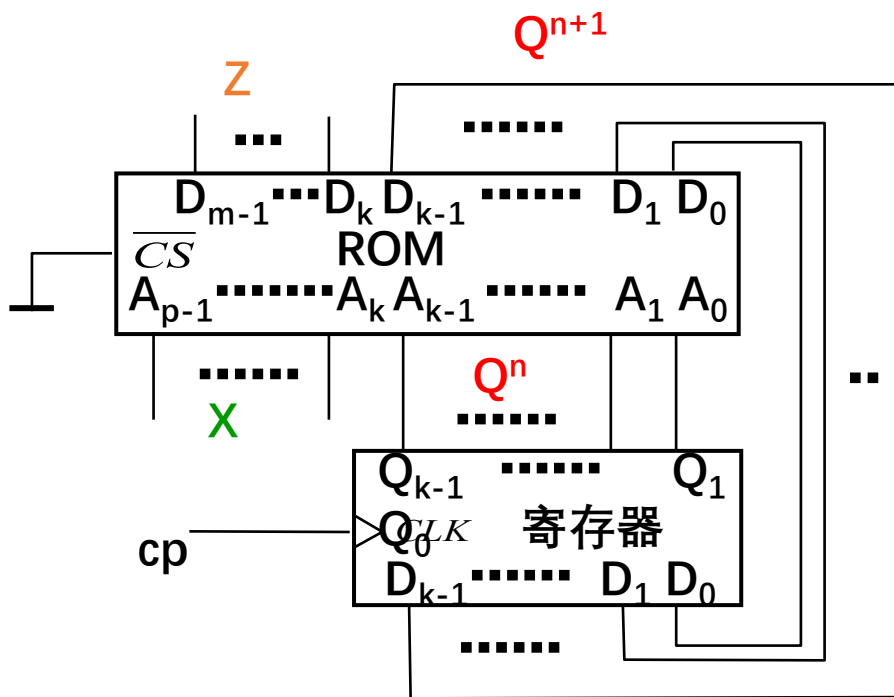
(2) 存储器实现时序逻辑

时序逻辑的激励函数 Y 、下一状态 Q^{n+1} 、输出函数 Z 都是输入信号 X 和当前状态 Q^n 的组合逻辑函数，意味着利用存储器可实现同步时序逻辑。



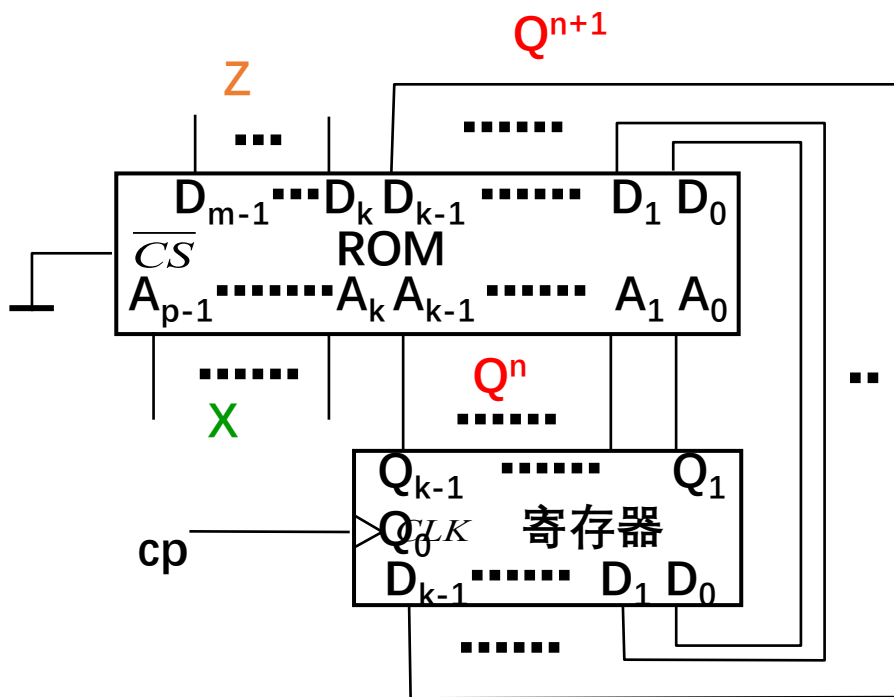
寄存器是为了使状态变化与时钟 cp 的有效边沿同步，如果ROM能做到边沿使能，就可不用寄存器。

7.1.4 用存储器实现逻辑处理(八)



- 当前状态码 $Q_i^n (i=0 \sim k-1)$ 和外输入 X 送到 ROM 的地址，以外输入 X 和 Q_i^n 为地址所寻址的存储单元内总存放下一状态的状态码 $Q_i^{n+1} (i=0 \sim k-1)$ 和输出 Z ，则下一状态函数 $Q^{n+1} = h(X, Q^n)$ 及输出函数 $Z = g(X, Q^n)$ 均由 ROM 实现。
- 实现的方法就是根据 Q^{n+1} 和 Z 的真值表将数据写入以当前状态码 Q^n 和外输入 X 为地址的存储单元。

7.1.4 用存储器实现逻辑处理(九)



- 在每个时钟的有效沿，寄存器输出更新，上一状态被作为了当前状态 Q^n 。ROM以当前输入 X 和 Q^n 作为地址去寻址输出 Z 和 Q^{n+1} 。 Z 作为时序电路的当前处理结果直接输出， Q^{n+1} 作为对处理阶段的记忆保存到下一时钟有效沿时刻。
- 此结构既可实现米里(Mealy)型也可实现摩尔(Moore)型时序电路。

7.1.4 用存储器实现逻辑处理(十)

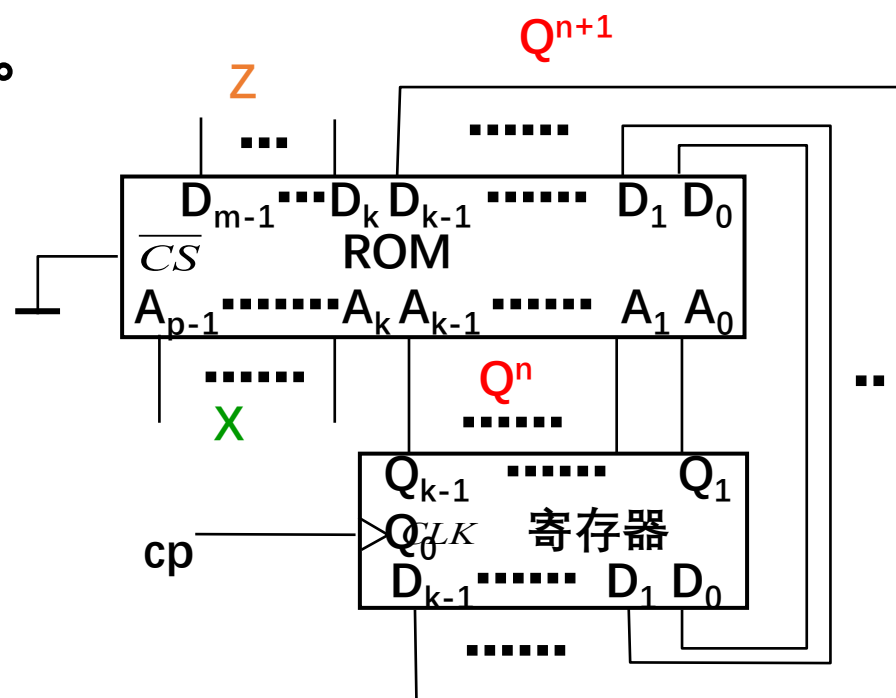
例7.1.2：实现8421码模10加法计数器，有1位输出Z，Z在状态为1001时，输出1，其它状态时输出0

由于计数模值为10，需要4位状态码，故 $k=4$ 。

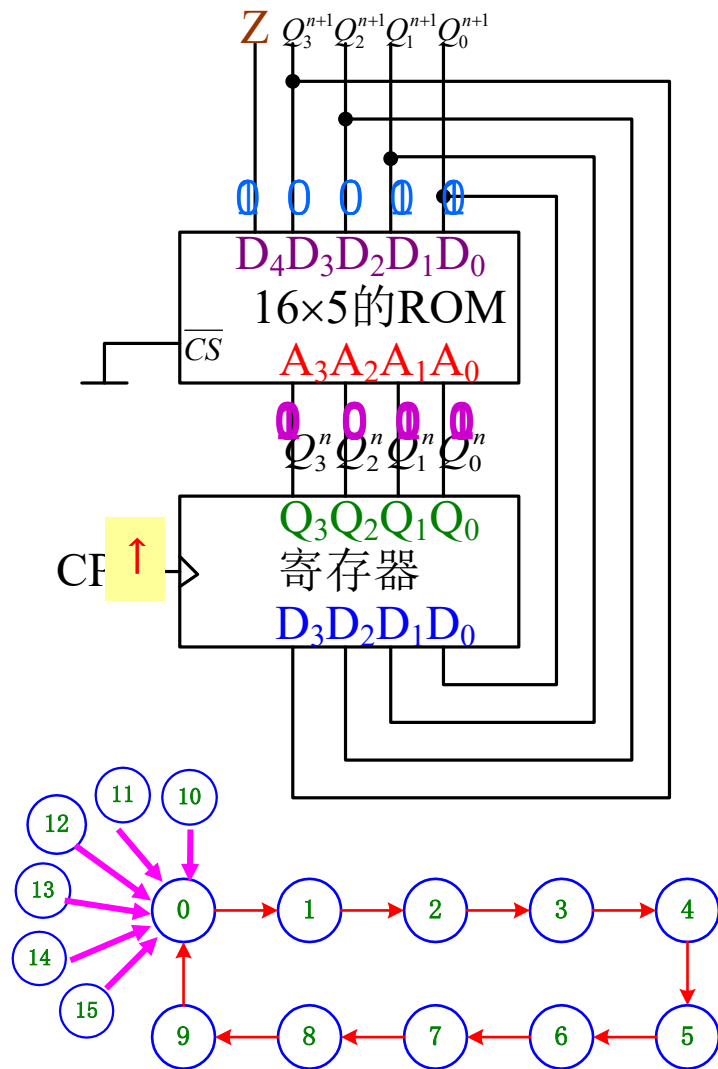
没有外输入变量X，ROM仅需4条地址线，故 $p=k=4$ 。

每个存储单元需5个存储位(1位输出Z + 4位状态码 $Q_i^n(i=0\sim 3)$), 故 $m=5$ 。

需用10个存储单元保存10个状态值，4位地址的ROM有16个存储单元。



7.1.4 用存储器实现逻辑处理(十一)



A_3	A_2	A_1	A_0	D_4	D_3	D_2	D_1	D_0
Q_3^n	Q_2^n	Q_1^n	Q_0^n	Z	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1	0
0	0	1	0	0	0	0	1	1
0	0	1	1	0	0	1	0	0
0	1	0	0	0	0	1	0	1
0	1	0	1	0	0	1	1	0
0	1	1	0	0	0	1	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	0	1	0	0	1
1	0	0	1	1	0	0	0	0
1	0	1	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0
1	1	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0
1	1	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0

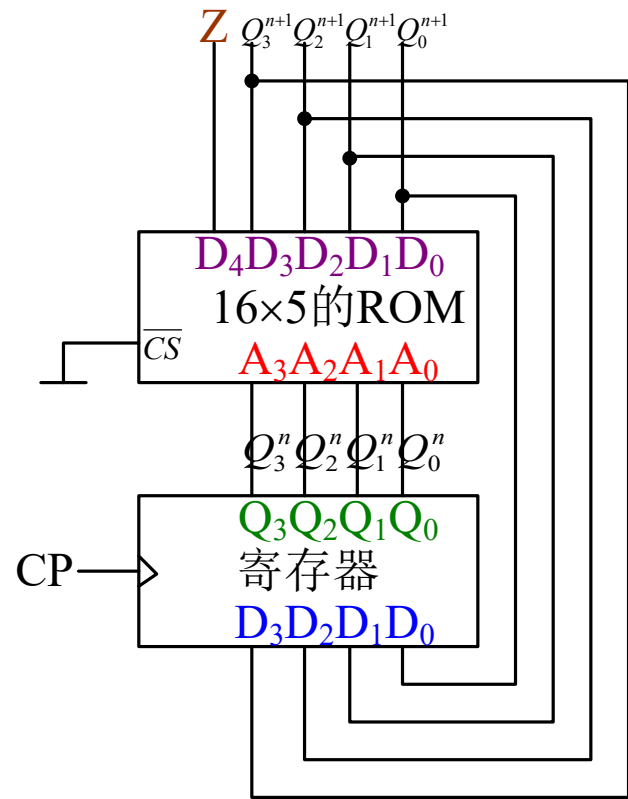
ROM
中存储
的数据

7.1.4 用存储器实现逻辑处理(十二)

存储器实现序列信号发生器

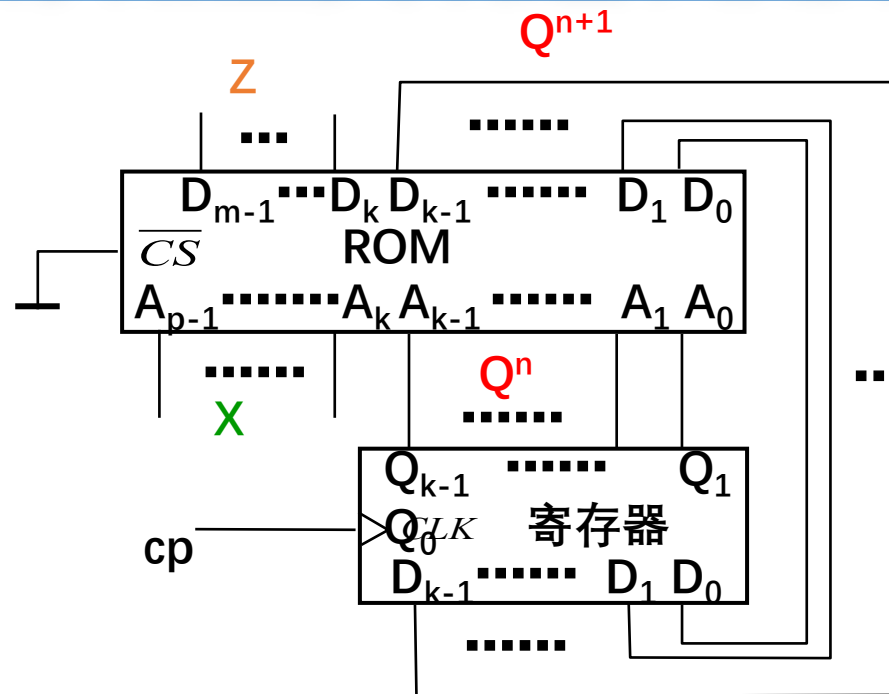
用此电路模板既可实现计数型的也可实现移存型的序列信号发生器。

在设计计数型序列信号发生器时，首先由序列码长度 M 确定地址位数 k ，随后设计模值为 M 的计数器，计数器的码型可任选。然后依次将序列信号值和下一状态码值写入以当前状态为地址的存储单元。



例如在实现 $M=10$ 的单一序列0110100011时，可用与例7.1.2相同的电路，将序列码值由前至后依次填入表7.1.2的 $D_4(Z)$ 存储位。工作时，由 $D_4(Z)$ 的输出可得到所要求的序列。

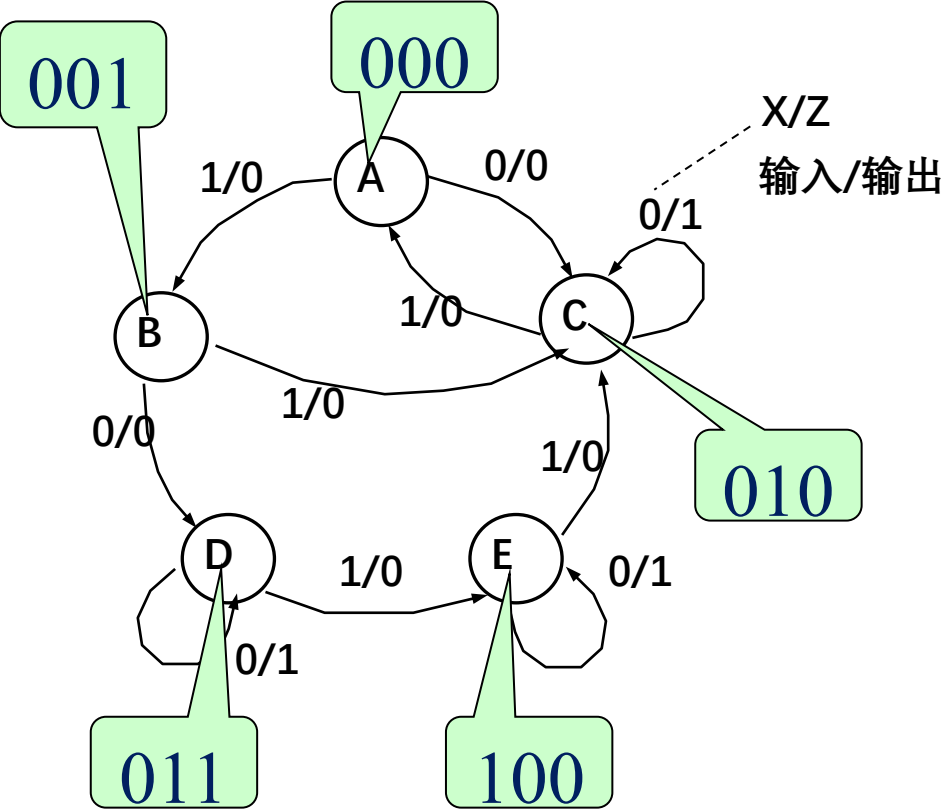
7.1.4 用存储器实现逻辑处理(十三)



设计移存型序列发生器时，首先确定所需的存储位的数目 k 和状态转移表。然后，根据状态转移表，依次将下一状态码填入以当前状态码为地址的存储单元。ROM存储单元的数目应等于或大于序列码的长度 M 。由 k 位数据输出端的任何一位都可得到所要求的序列信号。

7.1.4 用存储器实现逻辑处理(十四)

例7.1.3：基于ROM，实现如图所示的状态图



状态编码

状态转移表

ROM地址				数据 线			
A_3	A_2	A_1	A_0	D_3	D_2	D_1	D_0
$Q_2^n Q_1^n Q_0^n X$				$Z Q_2^{n+1} Q_1^{n+1} Q_0^{n+1}$			
0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	1	0	1	0
0	1	0	1	0	0	0	0
0	1	1	0	1	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	1	0	0	0	0

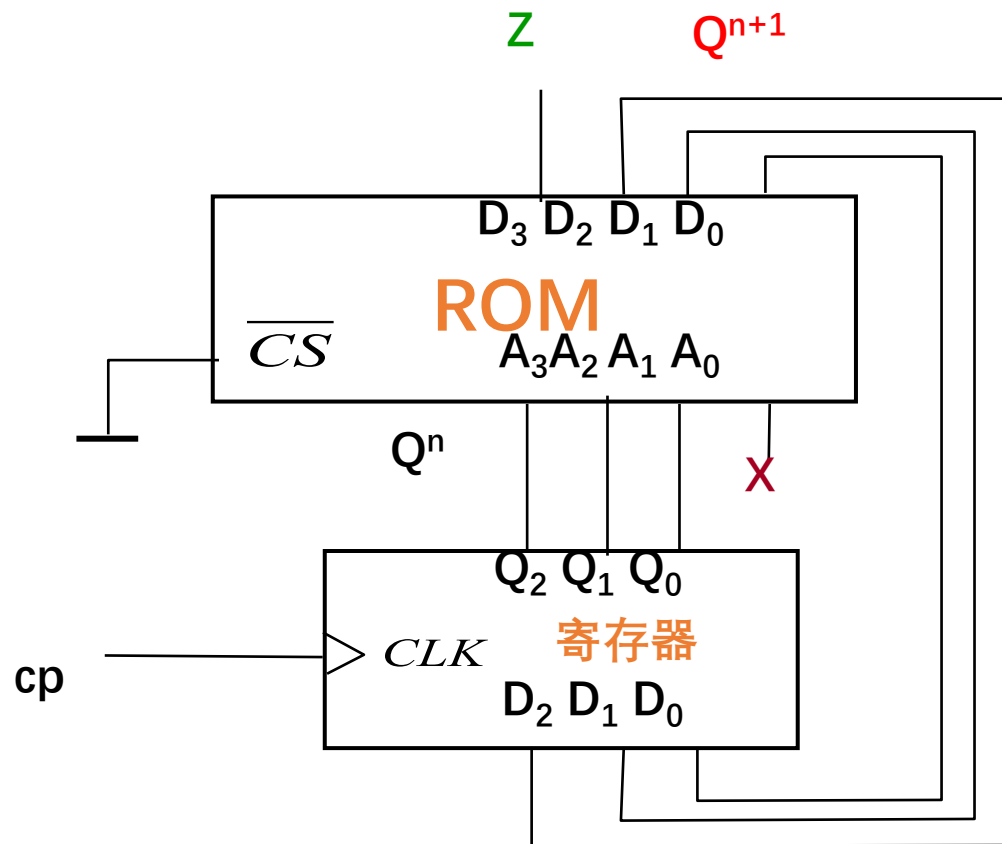
ROM中存储的数据

7.1.4 用存储器实现逻辑处理(十五)

ROM地址 数据 线
 $A_3 A_2 A_1 A_0$ $D_3 D_2 D_1 D_0$

$Q_2^n Q_1^n Q_0^n X$	$Z Q_2^{n+1} Q_1^{n+1} Q_0^{n+1}$
0 0 0 0	0 0 1 0
0 0 0 1	0 0 0 1
0 0 1 0	0 0 1 1
0 0 1 1	0 0 1 0
0 1 0 0	1 0 1 0
0 1 0 1	0 0 0 0
0 1 1 0	1 0 1 1
0 1 1 1	0 1 0 0
1 0 0 0	1 1 0 0
1 0 0 1	0 0 1 0
1 0 1 0	0 0 0 0
1 0 1 1	0 0 0 0
1 1 0 0	0 0 0 0
1 1 0 1	0 0 0 0
1 1 1 0	0 0 0 0
1 1 1 1	0 0 0 0

画电路图



小结

PLD的特点：可以通过编程的方法来设置其逻辑功能。

低密度PLD：PROM、PLA、PAL和GAL

高密度PLD：EPLD、CPLD和FPGA

PROM、PLA、PAL、GAL和EPLD实现组合逻辑函数的原理是任何组合逻辑电路均可表示为与或表达式，再加上触发器就可以实现时序电路。